
Association Schemes

Release 1.0

Sarobidy Razafimahatratra

Jun 15, 2026

CONTENTS:

1	Association Schemes	1
1.1	Generic association schemes	1
1.2	Association Schemes with names	8
2	Permutation Groups	11
	Python Module Index	13
	Index	15

ASSOCIATION SCHEMES

1.1 Generic association schemes

1.1.1 AssociationSchemes.generic_schemes module

class AssociationSchemes.generic_schemes.AssociationScheme(*gens*)

Bases: object

P_matrix()

Return the first eigenmatrix of *self*. This is the same as *character_table()*.

Q_matrix()

Return the first eigenmatrix of *self*.

EXAMPLE:

```
sage: X = graphs.ShrikhandeGraph()
sage: G = X.automorphism_group()
sage: A = X.adjacency_matrix()
sage: B = X.complement().adjacency_matrix()
sage: I = matrix.identity(X.order())
sage: AS = AssociationScheme([I,A,B])
sage: AS.Q_matrix()
[ 1  6  9]
[ 1  2 -3]
[ 1 -2  1]
sage: AS1 = OrbitalSchemeTransitiveGroup(G)
sage: AS1.Q_matrix()
[ 1  6  3  6]
[ 1  2 -1 -2]
[ 1 -2 -1  2]
[ 1 -2  3 -2]
```

TerwilligerAlgebra(*v*, *ring*=<cyfunction ComplexField>)

Return the Terwilliger algebra, over *ring*, of *self* with respect to *vertex*.

EXAMPLE:

```
sage: X = graphs.ShrikhandeGraph()
sage: G = X.automorphism_group()
sage: AS = OrbitalSchemeTransitiveGroup(G)
sage: T = AS.TerwilligerAlgebra(1,ring=CC)
```

(continues on next page)

(continued from previous page)

```
sage: T
Free module generated by {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15,
↪16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30} over Complex
↪Field with 53 bits of precision
sage: T.dimension()
31
```

adjacency_algebra(*ring*)

Return the adjacency algebra of *self* over the commutative ring *ring*. That is, the algebra generated by the adjacency matrices, over the commutative algebra *ring*.

EXAMPLE:

```
sage: X = graphs.ShrikhandeGraph()
sage: A = X.adjacency_matrix()
sage: B = X.complement().adjacency_matrix()
sage: I = matrix.identity(X.order())
sage: AS = AssociationScheme([I,A,B])
sage: AS.adjacency_algebra(ring=CC)
Free module generated by {0, 1, 2} over Complex Field with 53 bits of precision
sage: AS.adjacency_algebra(ring=ZZ)
Free module generated by {0, 1, 2} over Integer Ring
sage: AS.adjacency_algebra(ring=QQ)
Free module generated by {0, 1, 2} over Rational Field
```

adjacency_matrices()

Return the adjacency matrices of *self* as a list.

```
sage: X = graphs.PetersenGraph()
sage: A = X.adjacency_matrix()
sage: B = X.complement().adjacency_matrix()
sage: I = matrix.identity(X.order())
sage: AS = AssociationScheme([I,A,B])
sage: AS.adjacency_matrices()
[
[1 0 0 0 0 0 0 0 0 0] [0 1 0 0 1 1 0 0 0 0] [0 0 1 1 0 0 1 1 1 1]
[0 1 0 0 0 0 0 0 0 0] [1 0 1 0 0 0 1 0 0 0] [0 0 0 1 1 1 0 1 1 1]
[0 0 1 0 0 0 0 0 0 0] [0 1 0 1 0 0 0 1 0 0] [1 0 0 0 1 1 1 0 1 1]
[0 0 0 1 0 0 0 0 0 0] [0 0 1 0 1 0 0 0 1 0] [1 1 0 0 0 1 1 1 0 1]
[0 0 0 0 1 0 0 0 0 0] [1 0 0 1 0 0 0 0 0 1] [0 1 1 0 0 1 1 1 1 0]
[0 0 0 0 0 1 0 0 0 0] [1 0 0 0 0 0 0 0 1 1] [0 1 1 1 1 0 1 0 0 1]
[0 0 0 0 0 0 1 0 0 0] [0 1 0 0 0 0 0 0 1 1] [1 0 1 1 1 1 0 1 0 0]
[0 0 0 0 0 0 0 1 0 0] [0 0 1 0 0 1 0 0 0 1] [1 1 0 1 1 0 1 0 1 0]
[0 0 0 0 0 0 0 0 1 0] [0 0 0 1 0 1 1 0 0 0] [1 1 1 0 1 0 0 1 0 1]
[0 0 0 0 0 0 0 0 0 1], [0 0 0 0 1 0 1 1 0 0], [1 1 1 1 0 1 0 0 1 0]
]
```

automorphism_group()

Return the automorphism group of *self*, that is, the permutation group that preserves all relations of *self*.

EXAMPLE:

```

sage: X = graphs.ShrikhandeGraph()
sage: G = X.automorphism_group()
sage: A = X.adjacency_matrix()
sage: B = X.complement().adjacency_matrix()
sage: I = matrix.identity(X.order())
sage: AS = AssociationScheme([I,A,B])
sage: K = AS.automorphism_group()
sage: K.structure_description()
'(((C4 x C4) : C3) : C2) : C2'
sage: K.is_transitive()
True
    
```

base_matrix()

Return the base matrix of self. If $(\Omega, \mathcal{R})$ is an association scheme with adjacency matrices $A_0 = I, A_1, \dots, A_d$, then the *base matrix* of $(\Omega, \mathcal{R})$ is the matrix $0A_0 + 1A_1 + 2A_2 + \dots + dA_d$.

```

sage: X = graphs.PetersenGraph()
sage: A = X.adjacency_matrix()
sage: B = X.complement().adjacency_matrix()
sage: I = matrix.identity(X.order())
sage: AS = AssociationScheme([I,A,B])
sage: AS.base_matrix()
[0 1 2 2 1 1 2 2 2 2]
[1 0 1 2 2 2 1 2 2 2]
[2 1 0 1 2 2 2 1 2 2]
[2 2 1 0 1 2 2 2 1 2]
[1 2 2 1 0 2 2 2 2 1]
[1 2 2 2 2 0 2 1 1 2]
[2 1 2 2 2 2 0 2 1 1]
[2 2 1 2 2 1 2 0 2 1]
[2 2 2 1 2 1 1 2 0 2]
[2 2 2 2 1 2 1 1 2 0]
    
```

bose_mesner_algebra()

Return the Bose-Mesner algebra of the association scheme. See also `adjacency_algebra()`.

character_table()

Return the first eigenmatrix of self.

EXAMPLE:

```

sage: X = graphs.ShrikhandeGraph()
sage: G = X.automorphism_group()
sage: A = X.adjacency_matrix()
sage: B = X.complement().adjacency_matrix()
sage: I = matrix.identity(X.order())
sage: AS = AssociationScheme([I,A,B])
sage: AS.character_table()
[ 1  6  9]
[ 1  2 -3]
[ 1 -2  1]
sage: AS1 = OrbitalSchemeTransitiveGroup(G)
sage: AS1.character_table()
    
```

(continues on next page)

(continued from previous page)

```
[ 1  6  6  3]
[ 1  2 -2 -1]
[ 1 -2 -2  3]
[ 1 -2  2 -1]
```

dimension_of_centralizer_algebra(*v*, *matrix=False*)

Return the dimension of the centralizer algebra of the stabilizer of *vertex* in the automorphism group of *self* if *matrix=False*. If *matrix=True*, then it returns the block dimension decomposition of the centralizer algebra.

EXAMPLE:

```
sage: X = graphs.ShrikhandeGraph()
sage: G = X.automorphism_group()
sage: AS = OrbitalSchemeTransitiveGroup(G)
sage: AS.dimension_of_t_zero()
sage: AS.dimension_of_centralizer_algebra(1)
31
sage: AS.dimension_of_centralizer_algebra(1,matrix=True)
[1 1 1 1]
[1 4 2 3]
[1 2 2 2]
[1 3 2 4]
```

dimension_of_t_zero(*matrix=False*)

Return the dimension of the subspace T_0 of the Terwilliger algebra with respect to any vertex.

EXAMPLE:

```
sage: X = graphs.ShrikhandeGraph()
sage: G = X.automorphism_group()
sage: AS = OrbitalSchemeTransitiveGroup(G)
sage: AS.dimension_of_t_zero()
31
sage: AS.dimension_of_t_zero(matrix=True)
[1 1 1 1]
[1 4 3 2]
[1 3 4 2]
[1 2 2 2]
```

fusion(*P*, *return_scheme=False*)

Return whether the partition *P* of the vertices is an association scheme.

EXAMPLE:

```
sage: X = graphs.ShrikhandeGraph()
sage: G = X.automorphism_group()
sage: G = PermutationGroup(G.gens())
sage: AS = OrbitalSchemeTransitiveGroup(G)
sage: L = AS.adjacency_matrices()
sage: L[3]
[1 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 1 0 0 0 0 0 0 0 0 0 0 0 0]
```

(continues on next page)

(continued from previous page)

```

[0 0 1 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 1 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 1 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 1 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 1 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 1 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 1 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 1 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 1 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 1 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 1 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 1 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 1]
sage: AS.fusion([[0],[1,2,3]])
False
sage: AS.fusion([[3],[0,1,2]])
True
sage: AS.fusion([[3],[0,1,2]],return_scheme=True)
(True, <__main__.AssociationScheme object at 0x720021e86890>)
sage: AS.fusion([[3],[0,1],[2]],return_scheme=True)
(True, <__main__.AssociationScheme object at 0x720022477e70>)

```

graphs_in_scheme(digraphs=False)

Return the graphs corresponding to symmetric classes of self if digraphs = False, otherwise, all digraphs of the association scheme.

EXAMPLE:

```

sage: X = graphs.ShrikhandeGraph()
sage: G = X.automorphism_group()
sage: AS = OrbitalSchemeTransitiveGroup(G)
sage: AS.graphs_in_scheme()
[Graph on 16 vertices, Graph on 16 vertices, Graph on 16 vertices]

```

inner_distribution(Y)

intersection_number(i, j, k)

Return the intersection number p_{ij}^k of the association scheme self.

INPUT: integers i, j , and k between 0 and the r , where $r + 1$ is the rank of the association scheme.

OUTPUT: the value of p_{ij}^k .

EXAMPLE:

For example, the intersection numbers of the affine polar graph $VO_6^-(2)$ can be computed as follows.

```

sage: X = graphs.AffineOrthogonalPolarGraph(6,2,sign="-")
sage: A = X.adjacency_matrix()
sage: B = X.complement().adjacency_matrix()
sage: I = matrix.identity(X.order())
sage: AS = AssociationScheme([I,A,B])
sage: AS.intersection_number(0,1,1)
1

```

(continues on next page)

(continued from previous page)

```
sage: AS.intersection_number(1,1,1)
10
sage: AS.intersection_number(2,2,1)
20
sage: AS.intersection_number(2,2,2)
20
```

`is_AssociationScheme()`

Return whether `self` is an association scheme.

EXAMPLE:

```
sage: AS = JohnsonScheme(7,3)
sage: I,A,B,C = AS.adjacency_matrices()
sage: AS1 = AssociationScheme([I,A+B+C])
sage: AS1.is_AssociationScheme()
True
sage: AS2 = AssociationScheme([I,A+B])
sage: AS2.is_AssociationScheme()
False
sage: AS3 = AssociationScheme([I,A+B,C])
sage: AS3.is_AssociationScheme()
False
```

`is_commutative()`

Return whether or not `self` is a commutative association scheme.

The d -class association scheme $(\Omega, \mathcal{R})$ is commutative if its intersection numbers satisfy $p_{ij}^k = p_{ji}^k$, for all $0 \leq i, j, k \leq d$.

EXAMPLE:

```
sage: AS = OrbitalSchemeTransitiveGroup(group_acting_on_subsets(PSL(2,7),2))
sage: AS.is_commutative()
False
sage: AS = OrbitalSchemeTransitiveGroup(group_acting_on_
↳ subsets(AlternatingGroup(7),2))
sage: AS.is_commutative()
True
```

`is_formally_self_dual()`

Return whether `self` is formally self dual. That is, whether $Q = \overline{\{P\}}$.

EXAMPLE:

```
sage: X = graphs.ShrikhandeGraph()
sage: G = X.automorphism_group()
sage: A = X.adjacency_matrix()
sage: B = X.complement().adjacency_matrix()
sage: I = matrix.identity(X.order())
sage: AS = AssociationScheme([I,A,B])
sage: AS1 = OrbitalSchemeTransitiveGroup(G)
sage: AS.is_formally_self_dual()
True
```

(continues on next page)

(continued from previous page)

```
sage: AS1.is_formally_self_dual()
False
```

is_schurian()

Return whether or not `self` is Schurian, that is, its relations are the orbitals of a transitive group

EXAMPLE:

```
sage: X = graphs.ShrikhandeGraph()
sage: G = X.automorphism_group()
sage: AS1 = OrbitalSchemeTransitiveGroup(G)
sage: AS1.is_schurian()
True
sage: A = X.adjacency_matrix()
sage: B = X.complement().adjacency_matrix()
sage: I = matrix.identity(X.order())
sage: AS2 = AssociationScheme([I,A,B])
sage: AS2.is_schurian()
False
```

is_triply_regular()

Return whether or not the association scheme is triply regular.

EXAMPLE:

```
sage: X = graphs.HigmanSimsGraph()
sage: A = X.adjacency_matrix()
sage: B = X.complement().adjacency_matrix()
sage: I = matrix.identity(X.order())
sage: AS = AssociationScheme([I,A,B])
sage: AS.is_triply_regular()
True
```

krein_parameters(i, j, k)

Return the value of the Krein parameter q_{ij}^k .

INPUT: integers i, j , and k between 0 and the r , where $r + 1$ is the rank of the association scheme.

OUTPUT: the value of q_{ij}^k .

EXAMPLE:

```
sage: AS = HammingScheme(5,2)
sage: AS.krein_parameters(1,1,1)
0
sage: AS.krein_parameters(2,2,2)
0
sage: AS.krein_parameters(2,2,1)
0
sage: AS.krein_parameters(2,2,0)
5
```

order()

Return the number of vertices in `self`.

```
sage: X = graphs.PetersenGraph()
sage: A = X.adjacency_matrix()
sage: B = X.complement().adjacency_matrix()
sage: I = matrix.identity(X.order())
sage: AS = AssociationScheme([I,A,B])
sage: AS.order()
10
```

rank()

Return the number of relations in self.

```
sage: X = graphs.PetersenGraph()
sage: A = X.adjacency_matrix()
sage: B = X.complement().adjacency_matrix()
sage: I = matrix.identity(X.order())
sage: AS = AssociationScheme([I,A,B])
sage: AS.rank()
3
```

ratio_bound(i)

Return the value of Hoffman's ratio bound for the i-th graph, if it is symmetric.

EXAMPLE:

```
sage: AS = JohnsonScheme(8,3)
sage: AS.ratio_bound(0)
'... the index needs to be larger than 0'
sage: AS.ratio_bound(1)
28/3
sage: AS.ratio_bound(2)
8
sage: AS.ratio_bound(3)
21
```

AssociationSchemes.generic_schemes.**Schur_multiplication**(*A, B, ring=<class 'sage.rings.rational_field.RationalField'>*)

AssociationSchemes.generic_schemes.**base_matrix_to_adjacency_matrices**(*M*)

AssociationSchemes.generic_schemes.**common_eigenvectors**(*L*)

AssociationSchemes.generic_schemes.**spectral_decomposition_of_matrix**(*A*)

1.2 Association Schemes with names

1.2.1 AssociationSchemes.perfect_matching module

AssociationSchemes.perfect_matching.**PerfectMatchingScheme**(*k*)

Return the perfect matching association scheme with parameter equal to *self*.

EXAMPLE:

```

sage: X = graphs.ShrikhandeGraph()
sage: G = X.automorphism_group()
sage: A = X.adjacency_matrix()
sage: B = X.complement().adjacency_matrix()
sage: I = matrix.identity(X.order())
sage: AS = AssociationScheme([I,A,B])
sage: AS1 = OrbitalSchemeTransitiveGroup(G)
sage: AS.is_formally_self_dual()
True
sage: AS1.is_formally_self_dual()
False

```

`AssociationSchemes.perfect_matching.cycle_type_of_perfect_matching( $P, Q$ )`

`AssociationSchemes.perfect_matching.is_even_partition( $p$ )`

1.2.2 AssociationSchemes.schemes_with_names module

`AssociationSchemes.schemes_with_names.GrassmannScheme( $q, n, k$ )`

`AssociationSchemes.schemes_with_names.GroupScheme( $G$ )`

`AssociationSchemes.schemes_with_names.HammingScheme( $D, q$ )`

`AssociationSchemes.schemes_with_names.JohnsonScheme( $n, k$ )`

`AssociationSchemes.schemes_with_names.OrbitalSchemeGroupAction( $G$ )`

Return the orbital scheme of the transitive group G .

INPUT: G - a transitive group.

OUTPUT: orbital scheme of the transitive group G .

`AssociationSchemes.schemes_with_names.OrbitalSchemeTransitiveGroup( $G$ )`

Return the orbital scheme of the transitive group G .

INPUT: G - a transitive group.

OUTPUT: orbital scheme of the transitive group G .

PERMUTATION GROUPS

```
class AssociationSchemes.permutation_group.PermutationGroup(gens=None, gap_group=None,
                                                            canonicalize=True, domain=None,
                                                            category=None)
```

Bases: `PermutationGroup_generic`

`action_on_subsets(k)`

`group_action(H)`

`is_core_free(H)`

`is_derangement(x)`

Return whether self is a derangement.

EXAMPLE:

```
sage: G = PermutationGroup(SymmetricGroup(5).gens())
sage: G.is_derangement(G((1,5,2)))
False
sage: G.is_derangement(G("(1,5,2)(3,4)"))
True
```

`is_quasi_primitive()`

`number_of_derangements()`

`orbital_digraphs()`

`permutation_character()`

`pointwise_stabilizer(S)`

`rank_of_group()`

`setwise_stabilizer(S)`

`stabilizer_of_invariant_partition(L)`

`sub_orbits(v)`

PYTHON MODULE INDEX

a

`AssociationSchemes.generic_schemes`, 1
`AssociationSchemes.perfect_matching`, 8
`AssociationSchemes.permutation_group`, 11
`AssociationSchemes.schemes_with_names`, 9

INDEX

A

`action_on_subsets()` (AssociationSchemes.permutation_group.PermutationGroup method), 11

`adjacency_algebra()` (AssociationSchemes.generic_schemes.AssociationScheme method), 2

`adjacency_matrices()` (AssociationSchemes.generic_schemes.AssociationScheme method), 2

`AssociationScheme` (class in AssociationSchemes.generic_schemes), 1

`AssociationSchemes.generic_schemes` module, 1

`AssociationSchemes.perfect_matching` module, 8

`AssociationSchemes.permutation_group` module, 11

`AssociationSchemes.schemes_with_names` module, 9

`automorphism_group()` (AssociationSchemes.generic_schemes.AssociationScheme method), 2

B

`base_matrix()` (AssociationSchemes.generic_schemes.AssociationScheme method), 3

`base_matrix_to_adjacency_matrices()` (in module AssociationSchemes.generic_schemes), 8

`bose_mesner_algebra()` (AssociationSchemes.generic_schemes.AssociationScheme method), 3

C

`character_table()` (AssociationSchemes.generic_schemes.AssociationScheme method), 3

`common_eigenvectors()` (in module AssociationSchemes.generic_schemes), 8

`cycle_type_of_perfect_matching()` (in module AssociationSchemes.perfect_matching), 9

D

`dimension_of_centralizer_algebra()` (AssociationSchemes.generic_schemes.AssociationScheme method), 4

`dimension_of_t_zero()` (AssociationSchemes.generic_schemes.AssociationScheme method), 4

F

`fusion()` (AssociationSchemes.generic_schemes.AssociationScheme method), 4

G

`graphs_in_scheme()` (AssociationSchemes.generic_schemes.AssociationScheme method), 5

`GrassmannScheme()` (in module AssociationSchemes.schemes_with_names), 9

`group_action()` (AssociationSchemes.permutation_group.PermutationGroup method), 11

`GroupScheme()` (in module AssociationSchemes.schemes_with_names), 9

H

`HammingScheme()` (in module AssociationSchemes.schemes_with_names), 9

I

`inner_distribution()` (AssociationSchemes.generic_schemes.AssociationScheme method), 5

`intersection_number()` (AssociationSchemes.generic_schemes.AssociationScheme method), 5

`is_AssociationScheme()` (AssociationSchemes.generic_schemes.AssociationScheme method), 6

`is_commutative()` (AssociationSchemes.generic_schemes.AssociationScheme method), 6

[is_core_free\(\)](#) (AssociationSchemes.permutation_group.PermutationGroup method), 11
[is_derangement\(\)](#) (AssociationSchemes.permutation_group.PermutationGroup method), 11
[is_even_partition\(\)](#) (in module AssociationSchemes.perfect_matching), 9
[is_formally_self_dual\(\)](#) (AssociationSchemes.generic_schemes.AssociationScheme method), 6
[is_quasi_primitive\(\)](#) (AssociationSchemes.permutation_group.PermutationGroup method), 11
[is_schurian\(\)](#) (AssociationSchemes.generic_schemes.AssociationScheme method), 7
[is_triply_regular\(\)](#) (AssociationSchemes.generic_schemes.AssociationScheme method), 7

J

[JohnsonScheme\(\)](#) (in module AssociationSchemes.schemes_with_names), 9

K

[krein_parameters\(\)](#) (AssociationSchemes.generic_schemes.AssociationScheme method), 7

M

[module](#)
 AssociationSchemes.generic_schemes, 1
 AssociationSchemes.perfect_matching, 8
 AssociationSchemes.permutation_group, 11
 AssociationSchemes.schemes_with_names, 9

N

[number_of_derangements\(\)](#) (AssociationSchemes.permutation_group.PermutationGroup method), 11

O

[orbital_digraphs\(\)](#) (AssociationSchemes.permutation_group.PermutationGroup method), 11
[OrbitalSchemeGroupAction\(\)](#) (in module AssociationSchemes.schemes_with_names), 9
[OrbitalSchemeTransitiveGroup\(\)](#) (in module AssociationSchemes.schemes_with_names), 9
[order\(\)](#) (AssociationSchemes.generic_schemes.AssociationScheme method), 7

P

[P_matrix\(\)](#) (AssociationSchemes.generic_schemes.AssociationScheme method), 1
[PerfectMatchingScheme\(\)](#) (in module AssociationSchemes.perfect_matching), 8
[permutation_character\(\)](#) (AssociationSchemes.permutation_group.PermutationGroup method), 11
[PermutationGroup](#) (class in AssociationSchemes.permutation_group), 11
[pointwise_stabilizer\(\)](#) (AssociationSchemes.permutation_group.PermutationGroup method), 11

Q

[Q_matrix\(\)](#) (AssociationSchemes.generic_schemes.AssociationScheme method), 1

R

[rank\(\)](#) (AssociationSchemes.generic_schemes.AssociationScheme method), 8
[rank_of_group\(\)](#) (AssociationSchemes.permutation_group.PermutationGroup method), 11
[ratio_bound\(\)](#) (AssociationSchemes.generic_schemes.AssociationScheme method), 8

S

[Schur_multiplication\(\)](#) (in module AssociationSchemes.generic_schemes), 8
[setwise_stabilizer\(\)](#) (AssociationSchemes.permutation_group.PermutationGroup method), 11
[spectral_decomposition_of_matrix\(\)](#) (in module AssociationSchemes.generic_schemes), 8
[stabilizer_of_invariant_partition\(\)](#) (AssociationSchemes.permutation_group.PermutationGroup method), 11
[sub_orbits\(\)](#) (AssociationSchemes.permutation_group.PermutationGroup method), 11

T

[TerwilligerAlgebra\(\)](#) (AssociationSchemes.generic_schemes.AssociationScheme method), 1